

ETAPAS DEL DESARROLLO DE SOFTWARE: SISTEMA POWERTRACK PARA GIMNASIO

RESPUESTAS AL EJERCICIO DE SDLC Y
DISEÑO DE SOFTWARE

¿QUÉ ES SDLC Y SU OBJETIVO?

SDLC = Software Development Life Cycle

Es un proceso estructurado para crear software de calidad, abarcando desde la idea inicial hasta el mantenimiento post-lanzamiento. Es clave para organizar equipos, controlar tiempos y costos del proyecto.

LAS 7 FASES DEL SDLC

Fases 1-2

1. Planificación
2. Definir objetivos, alcance, recursos y cronograma del proyecto.
3. Análisis de Requisitos
4. Recopilar y documentar necesidades funcionales y no funcionales.

Fases 3-5

1. Diseño
2. Crear arquitectura y especificaciones técnicas del sistema.
3. Implementación
4. Codificar el software según el diseño.
5. Pruebas
6. Validar calidad y funcionamiento correcto.

Fases 6-7

1. Despliegue
2. Lanzar el sistema en el entorno de producción.
3. Mantenimiento
4. Corregir errores y evolucionar el software según nuevas necesidades.



IMPORTANCIA DEL ALCANCE

Definir bien el alcance del proyecto es fundamental para el éxito del desarrollo de software. Evita malentendidos entre el equipo y los stakeholders, permite controlar costos y tiempos de manera efectiva, establece expectativas claras desde el inicio y previene el temido "scope creep" — cambios fuera de control que descarrilan proyectos.

Sistema POWERTRACK

CLASIFICACIÓN DE REQUISITOS

REQUISITO A

FUNCIONAL

Iniciar sesión con correo electrónico y contraseña. El sistema debe permitir a los usuarios autenticarse mediante sus credenciales para acceder a las funcionalidades de POWERTRACK.

REQUISITO B

NO FUNCIONAL

El sistema debe responder en menos de 2 segundos. Este requisito de rendimiento garantiza una experiencia de usuario fluida y eficiente en el gimnasio.

CLASIFICACIÓN DE REQUISITOS

PARTE 2

Continuación del análisis de requisitos del sistema POWERTRACK, diferenciando entre funcionales y no funcionales para garantizar un desarrollo estructurado.

FUNCIONAL

c) Registrar pagos mensuales
El sistema debe permitir registrar y gestionar los pagos mensuales de membresías de los usuarios del gimnasio, incluyendo fecha, monto y método de pago.

NO FUNCIONAL

d) Soportar 500 usuarios simultáneos
El sistema debe mantener su rendimiento óptimo con al menos 500 usuarios conectados al mismo tiempo, garantizando escalabilidad y disponibilidad.

DISEÑO HLD VS LLD

HLD (ALTO NIVEL)

Arquitectura general del sistema. Define componentes principales y sus conexiones. Visión macro sin detalles de implementación. Responde: ¿Qué módulos existen y cómo se comunican?

LLD (BAJO NIVEL)

Diseño detallado de cada componente. Especifica algoritmos, estructuras de datos y lógica específica. Responde: ¿Cómo funciona internamente cada módulo?

ANALOGÍA

HLD = Plano general del edificio (ubicación de habitaciones, pasillos). LLD = Detalles de cada habitación (instalaciones eléctricas, materiales, acabados).

PATRÓN MVC



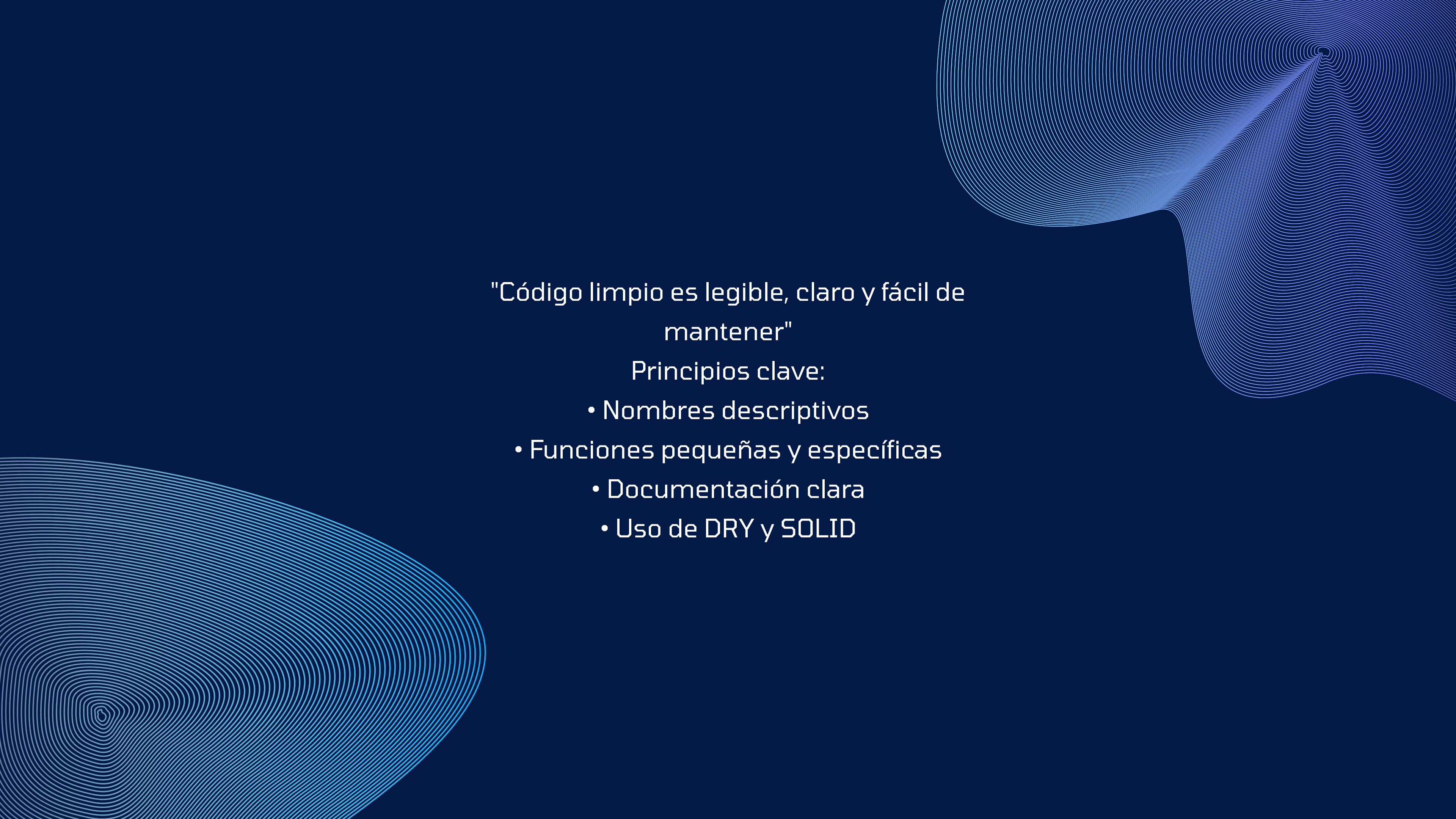
Modelo: Gestiona los datos y la lógica de negocio del sistema. Es el núcleo de la aplicación.



Vista: Interfaz gráfica que ve el usuario. Presenta los datos del Modelo de forma visual.



Controlador: Maneja la interacción del usuario y coordina Modelo y Vista. Procesa las solicitudes.



"Código limpio es legible, claro y fácil de mantener"

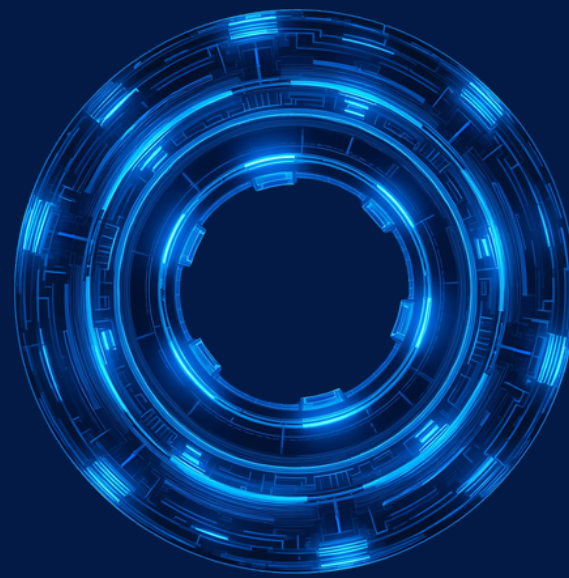
Principios clave:

- Nombres descriptivos
- Funciones pequeñas y específicas
 - Documentación clara
 - Uso de DRY y SOLID

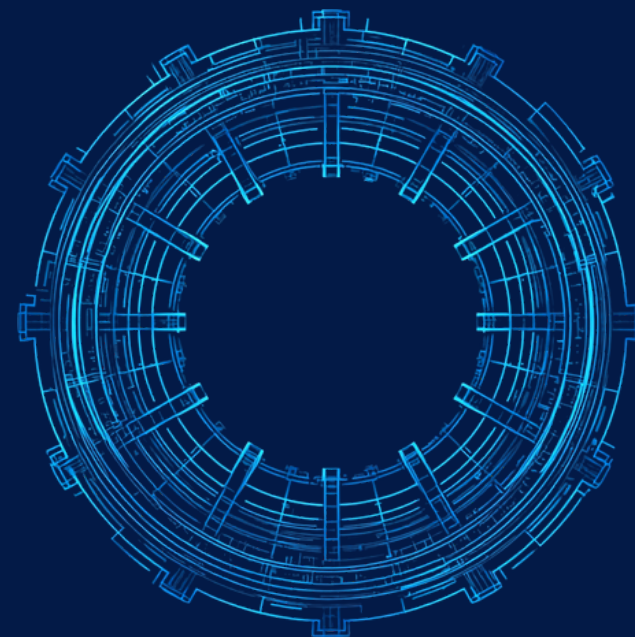
PRUEBAS UNITARIAS

Las pruebas unitarias validan funciones o métodos individuales del código, asegurando que cada componente funcione correctamente de forma aislada. Detectan errores en etapas tempranas del desarrollo, reduciendo costos de corrección. Son la base fundamental para pruebas automáticas y permiten refactorizaciones seguras del código.

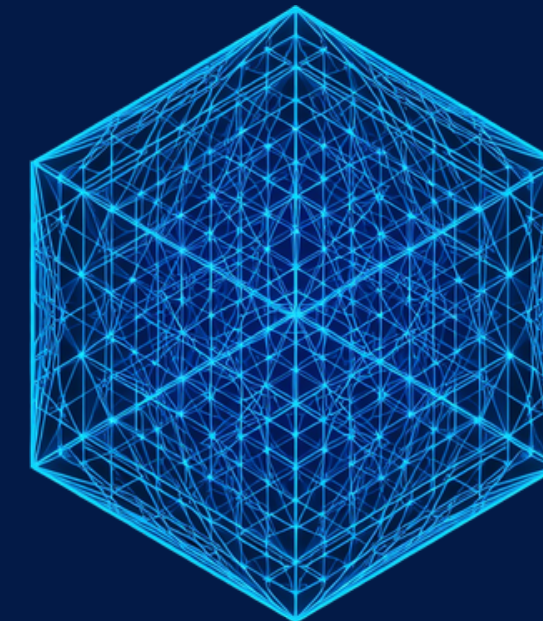
¿QUÉ ES CI/CD?



Integración Continua
Automatiza la integración de código de múltiples desarrolladores en un repositorio compartido constantemente.



Despliegue Continuo
Automatiza el proceso de llevar el código a producción, reduciendo errores humanos y acelerando entregas.



Desarrollo Ágil
Clave para ciclos de desarrollo rápidos, confiables y con retroalimentación constante del equipo.



DOCKER Y CONTENEDORES

Docker empaqueta aplicaciones junto con todas sus dependencias en contenedores portables y consistentes. Esto garantiza que el software funcione de manera idéntica en cualquier entorno, desde desarrollo hasta producción. Los contenedores son ligeros, rápidos de iniciar y permiten escalabilidad eficiente en la nube.

MANTENIMIENTO

CORRECTIVO

Corrige errores y bugs existentes en el sistema. Se activa cuando algo no funciona como debería. Objetivo: restaurar el funcionamiento correcto del software sin añadir nuevas capacidades.

EVOLUTIVO

Añade funcionalidades nuevas o mejora las existentes. Responde a cambios en necesidades del negocio o usuarios. Objetivo: hacer crecer y adaptar el sistema a nuevos requerimientos.

DIFERENCIAS

Correctivo es reactivo (responde a fallos), evolutivo es proactivo (anticipa necesidades). Ambos son esenciales para el ciclo de vida del software POWERTRACK y garantizan su longevidad.



ANÁLISIS FINAL

Análisis

Fase de Detección

Pruebas

Validación Requerida

Escalabilidad

Problema Detectado



GRACIAS